

HOW *TOP COMPANIES* USE SPRING BOOT



@java_in.depth

HOW TOP COMPANIES WRITE **SPRING BOOT**

The **actual patterns** Netflix, Uber, Amazon and fintechs use in production — circuit breakers, CQRS, outbox, observability, and more.

Project Structure

ARCHITECTURE

Circuit Breaker + Resilience4j

RESILIENCE

Structured Logging + Tracing

OBSERVABILITY

Global Exception Handling

API DESIGN

Idempotency Keys

PAYMENTS

Outbox Pattern

RELIABILITY

CQRS

SCALABILITY

Async + Virtual Threads

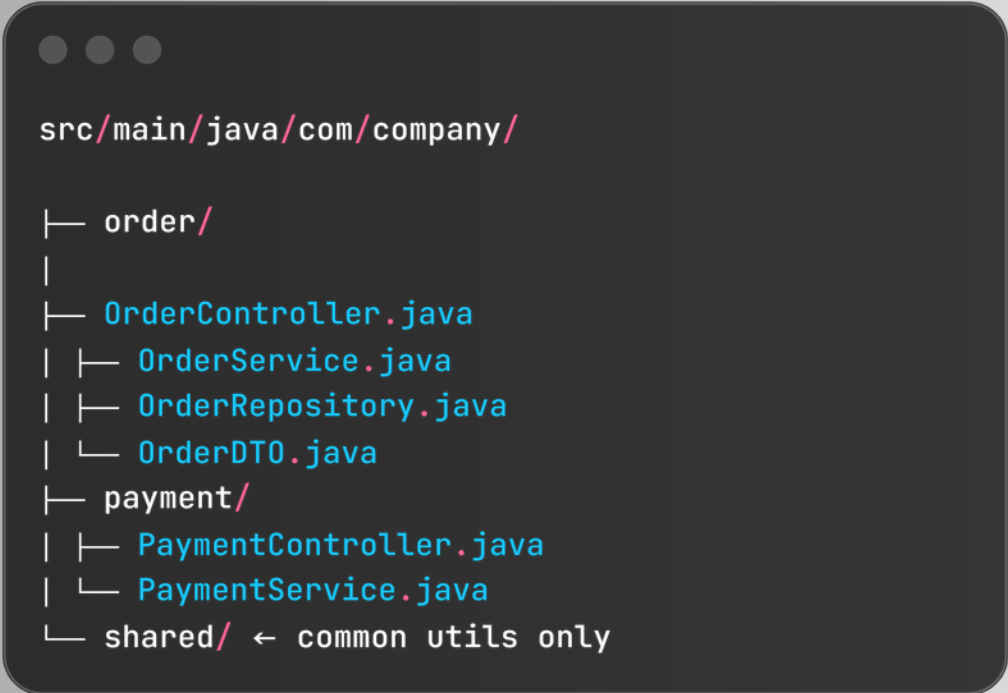
JAVA 21

PROJECT STRUCTURE

How code is organized before the first line is written

The Rule

Package by feature, not by layer. Not controller/service/repo folders — but order/, payment/, notification/ folders. Each owns its full vertical slice.



```
src/main/java/com/company/  
  
├─ order/  
│  
│ ├─ OrderController.java  
│ │ ├─ OrderService.java  
│ │ ├─ OrderRepository.java  
│ │ └─ OrderDTO.java  
│  
├─ payment/  
│ │ ├─ PaymentController.java  
│ │ └─ PaymentService.java  
└─ shared/ ← common utils only
```

No cross-feature imports

Order package doesn't import Payment internals. They talk through interfaces or events only

Spring Modulith enforces boundaries

New in Spring Boot 3 — catches illegal dependencies at test time, before they reach production

CIRCUIT BREAKER

RESILIENCE4J

Stop one failing service from crashing everything else

The Problem

Service B is down. Service A keeps hammering it. Threads pile up. Service A goes down. Now everything's down. One failure became a cascade.

```
@CircuitBreaker(name = "paymentCB", fallbackMethod =  
    "paymentFallback")  
@Retry(name = "paymentRetry")  
@TimeLimiter(name = "paymentTL")  
public PaymentResponse charge(PaymentRequest req) {  
    return paymentClient.charge(req);  
}  
  
// NEVER retry payments without idempotency key
```

3 states: CLOSED → OPEN → HALF-OPEN

Opens at 50% failure rate, waits 30s, then tests with limited requests before closing again

Don't retry non-idempotent calls

Retrying a payment = double charge. Only retry reads or calls protected by idempotency keys

STRUCTURED LOGGING + TRACING

You can't debug production without logs you can actually query

The Rule

Logs must be JSON. Every log must carry a traceId.
Without this, you can't correlate what happened across 10 microservices during an incident.

```
// BAD - unstructured
log.info("Order placed for user " + userId);

// GOOD - structured + trace
log.info("Order placed",
    kv("userId", userId),
    kv("orderId", orderId),
    kv("traceId", MDC.get("traceId")))
);

// {"level":"INFO","traceId":"abc123","userId":"u1","orderId":"o99","msg":"Order placed"}
```

OpenTelemetry + Micrometer

Spring Boot 3 auto-instruments traces. Spans flow across services automatically with B3/W3C headers

Stack: Logback + Logstash encoder → ELK / Loki.

Traces → Jaeger or Tempo. Metrics → Prometheus + Grafana

GLOBAL EXCEPTION HANDLING

Never expose a stack trace. Ever. Return structured, consistent errors.

The Rule

Every API error is a contract. Stripe's error format is so consistent developers can build on it blindly. That's what top companies aim for.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(OrderNotFoundException ex) {
        return ResponseEntity.status(404).body(
            new ApiError(
                "ORDER_NOT_FOUND",
                ex.getMessage(),
                MDC.get("traceId") // include traceId
            )
        );
    }
}
```

ApiError has 3 fields: code, message, traceId

Error code is machine-readable (ORDER_NOT_FOUND).

Message is human-readable. traceId links to logs

One handler, zero duplication

@RestControllerAdvice catches exceptions from every controller. No try-catch blocks in service layer

IDEMPOTENCY

KEYS

Same request, called twice — should produce the same result, not double the charge

The Problem

Network timeout. Client retries the payment request. Server already processed it. User is charged twice. Idempotency keys prevent this entirely.

```
// Client → Idempotency-Key header
POST /payments
Idempotency-Key: "uuid-abc-123"

// Server logic
String key = request.getHeader("Idempotency-Key");

if (redis.exists(key)) {
    return redis.get(key); // cached
}

PaymentResponse resp = processPayment(req);
redis.setex(key, 86400, resp); // 24h cache

return resp;
```

Rule: All non-GET endpoints that mutate money or data must support idempotency. Store key + response in Redis with TTL. Check before processing.

TRANSACTIONAL

OUTBOX PATTERN

Save to DB and publish to Kafka — atomically. No lost events, ever.

The Problem

You save order to DB, then try to publish to Kafka. Kafka call fails. DB has order. Kafka doesn't. Your systems are now inconsistent — and you'll never know.

```
// WRONG - not atomic
orderRepo.save(order);           // ✓
kafkaTemplate.send("orders", event); // ✗

// OUTBOX - single transaction
orderRepo.save(order);
outboxRepo.save(new OutboxEvent("ORDER_PLACED", payload)); // @Transactional

// Relay → reads outbox, publishes to Kafka
```

How it works: Save event to outbox table in same DB transaction. A separate relay process polls outbox and publishes to Kafka. At-least-once guaranteed.

CQRS

Separate how you write data from how you read it — different models, different databases

The Rule

Commands change state. Queries return data. Never both in the same method. Write to PostgreSQL (ACID). Read from Elasticsearch or Redis (fast, optimized for queries).

```
// COMMAND - change state
void placeOrder(PlaceOrderCommand cmd) {
    Order order = Order.from(cmd);
    orderRepo.save(order);
    eventBus.publish(new OrderPlacedEvent(order));
}

// QUERY - read only
OrderView getOrder(String orderId) {
    return orderReadRepo.findById(orderId); // Redis/ES
}
```

Read side = eventual consistency

Read model updates async via events. Data may be milliseconds stale. Design your UI to handle this

VIRTUAL THREADS **JAVA 21**

Handle 100,000+ concurrent requests — without reactive programming complexity

The Shift

Traditional threads: 1 thread per request, max ~500 concurrent. Virtual threads: JVM manages millions of lightweight threads. Your blocking code is now non-blocking — for free.

```
# application.yml — enable
spring:
  threads:
    virtual:
      enabled: true

// code stays same
@GetMapping("/orders")
public List<Order> getOrders() {
    return orderService.findAll(); // blocking OK
}
```

No ThreadLocal, no synchronized blocks

Virtual threads make ThreadLocal expensive. Top companies audit and remove them when enabling VT

WebFlux vs Virtual Threads

For most backends: Virtual Threads win on simplicity.
WebFlux only when backpressure or streaming is needed

SECURITY AT **API** LAYER

JWT alone isn't enough. Here's what production security actually looks like.

The Standard

Auth at gateway. Rate limiting per user. Secrets never in code. Audit log on every mutation. That's the minimum for a production API in 2026.

JWT + short expiry + refresh tokens

Access token: 15min. Refresh token: 7 days, stored HttpOnly. Never store tokens in localStorage

Secrets Manager, not application.yml

AWS Secrets Manager / Vault. DB passwords, API keys never in config files or env vars in plain text

Rate limiting at API Gateway level

Don't implement rate limiting in Spring — do it at gateway (AWS API Gateway, Nginx, Kong). One place, all services protected

THIS IS HOW PRODUCTION IS **WRITTEN.**

- 1 Package by feature
- 2 Circuit Breaker
- 3 Structured Logging
- 4 Global Exception Handler
- 5 Idempotency Keys
- 6 Outbox Pattern
- 7 CQRS
- 8 Virtual Threads (Java 21)
- 9 API Security

Save this. Pick one pattern per week. Implement it in a side project. In 9 weeks you write production Spring Boot like senior engineers do.

@java_in.depth